



ADVERTISEMENT FRAUD DETECTION USING MACHINE LEARNING



A PROJECT REPORT

Submitted by

KAMALESH M R (2105045)

KARNA C (2105067)

ANANDHA KRISHNAN P (2105069)

In partial fulfilment for the award of

the degree of

**BACHELOR OF
TECHNOLOGY**

in

DEPARTMENT OF INFORMATION TECHNOLOGY

SRI RAMAKRISHNA ENGINEERING COLLEGE, COIMBATORE

ANNA UNIVERSITY: CHENNAI 600025

May 2025

**SRI RAMAKRISHNA ENGINEERING COLLEGE
COIMBATORE**

ANNA UNIVERSITY: CHENNAI 600025

BONAFIDE CERTIFICATE

Certified that this project report entitled “**Advertisement Fraud Detection using Machine Learning**” is the Bonafide work of **KAMALESH M R (2105045)**, **KARNA C (2105067)**, **ANANDHA KRISHNAN P (2105069)**, who carried out the Project under my supervision.

SIGNATURE

SUPERVISOR

Mrs. M. Princy

Assistant Professor,

Department of Information Technology

Sri Ramakrishna Engineering College

Coimbatore - 641022

SIGNATURE

HEAD OF THE DEPARTMENT

Dr. M. Senthamil Selvi

Professor and Head

Department of Information Technology

Sri Ramakrishna Engineering College

Coimbatore – 641022

Submitted for Project Viva Voce Examination held on _____

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

We have immense pleasure in expressing our wholehearted thankfulness to **Shri. R. Sundar**, Managing Trustee, SNR Sons Charitable Trust and **Shri. S. Narendran**, Joint Managing Trustee, SNR Sons Charitable Trust for giving us the opportunity to study in our esteemed college and for very generously providing more than enough infrastructural facilities for us to get molded as a complete engineer.

We wish to express our profound and sincere gratitude to **Dr. A. Soundarrajan**, Principal, for inspiring us with his Engineering Wisdom. We also wish to record our heartfelt thanks for motivating and guiding us to become Industry ready engineers with inter-disciplinary knowledge and skillset with his multifaceted personality.

We extend our indebted thankfulness to **Dr. M. Senthamil Selvi**, Professor and Head, for guiding and helping us in all our activities to develop confidence and skills required to meet the challenges of the industry. We also express our gratitude for giving us support and guidance to complete the project duly.

We owe our deep gratitude to our project supervisor **Mrs. M. Princy**, Assistant Professor, Department of Information Technology who took keen interest in our project work and guided us all along with all the necessary inputs required to complete the project work.

We express our sincere thanks to our project coordinator **Dr. J. Angel Ida Chellam**, Associate Professor, Department of Information Technology, the evaluators teaching faculty members and supporting staff members of the department for evaluating the project and providing valuable suggestions for improvements.

ABSTRACT

Online advertising is essential to the field of digital marketing since it enables companies to effectively reach sizable and specific audiences. However, there are serious risks for both consumers and advertisers due to the quick increase in fraudulent ads. By inserting malicious content—such as phishing URLs, misleading pictures, and damaging video ads—within what appear to be genuine commercials, cybercriminals are increasingly taking advantage of online ad platforms. Serious repercussions from these misleading advertisements may include monetary loss, sensitive data theft, and a general deterioration in public confidence in digital advertising ecosystems.

This project suggests a clever and proactive method for identifying and stopping malicious or phishing ads on websites in order to address these issues. The method uses a carefully selected dataset of known harmful ad URLs and phishing sites to train a machine learning model. After training, the model is included into a browser plugin that tracks and evaluates online ads in real time while people are browsing the web. The plugin immediately stops potentially harmful advertisements when it detects them, shielding users from potentially harmful content.

By lessening user exposure to phishing and fraudulent adverts, this method seeks to improve internet security. By doing this, it helps rebuild confidence in online advertising platforms while simultaneously safeguarding user privacy and sensitive data. In the end, the project offers a scalable and effective way to counteract ad-based cyberthreats and encourage a safer online experience.

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	i
	LIST OF FIGURES	iii
	LIST OF ABBREVIATIONS	iv
1	INTRODUCTION	1
2	LITERATURE SURVEY	3
3	PROBLEM OVERVIEW	9
	3.1 PROBLEM IDENTIFICATION	9
	3.2 PROBLEM STATEMENT	11
	3.3 OBJECTIVE	11
	3.4 METHODOLOGY	12
	3.5 PLAN OF EXPERIMENTS	15
	3.6 SYSTEM SPECIFICATIONS	17
4	DESIGN AND MODELLING	19
5	RESULTS AND DISCUSSIONS	21
6	CONCLUSION AND FUTURE SCOPE	23
	6.1 CONCLUSION	23
	6.2 FUTURE SCOPE	24
7	REFERENCES	26
ANNEXURE 1	SOURCE CODE	28
	SNAPSHOTS	41

LIST OF FIGURES

FIGURES	TITLE	PAGE NO
3.4.1	Block diagram of Phishing Ads Detector	12
1.1	Extension UI of the Proposed System	41
1.2	Popup UI of the Proposed System	41
1.3	Output of the Proposed System	42

LIST OF ABBREVIATIONS

XGBOOST	Extreme Gradient Boost
KNN	K Nearest Neighbour
RFE	Recursive Feature Elimination
GB	Gradient Boosting
PPC	Pay Per Click
CNN	Convolutional Neural Network
DNN	Deep Neural Network
ANN	Artificial Neural Network
RNN	Recurrent Neural Network

CHAPTER 1

INTRODUCTION

Online advertising has become a key component of contemporary marketing tactics in the digital age, allowing companies to easily and effectively reach large audiences worldwide. Digital advertisements have revolutionized how businesses market their goods and services with sophisticated targeting tools, tailored content, and programmatic ad placements. Online advertisements, whether on websites, social media platforms, or search engines, are essential for raising brand awareness, bringing in visitors, and eventually making money.

The risks that target the digital advertising environment, however, are also constantly changing. The growing surge in fraudulent internet advertisements is one of the industry's most urgent problems at the moment. The use of ad networks by cybercriminals to disseminate malicious content masquerading as genuine adverts is growing. In order to trick users into clicking on them, these dangerous advertisements frequently include phishing URLs, deceptive images, or fake video content. Users' devices, financial information, and personal information may be seriously jeopardized if they are exposed to malware downloads, phony login pages, or scam websites. Digital platforms and genuine advertisers are also impacted by this increasing wave of ad-based cyberthreats. The repercussions for consumers can include malware and spyware installation, financial fraud, and identity theft. Fraudulent advertisements erode user trust, reduce engagement, and harm a brand's reputation, according to advertisers and platform providers. Furthermore, if user data is compromised by harmful advertisements, platforms may potentially have compliance problems with data protection laws like the CCPA or GDPR.

Intelligent systems that can recognize and block phishing or fraudulent adverts in real time are desperately needed, given the scope and severity of this problem. Our initiative offers a machine learning-based approach to address this issue by identifying hazardous internet advertisements before users interact with them. We developed a prediction model to identify and flag questionable material by using a carefully selected dataset of phishing ad links. After that, the trained model was made available as a browser extension, allowing users to easily and instantly identify phishing advertisements while they were online.

By keeping an eye on website content and warning users when a potentially harmful advertisement is found, this browser plugin serves as a proactive barrier. Our technology aims to contribute to a more reliable and secure digital advertising environment in addition to improving user safety and data protection. By incorporating intelligent threat detection into routine online browsing, this project ultimately seeks to close the gap between cybersecurity and user experience.

CHAPTER 2

LITERATURE SURVEY

[1] R. A. Alzahrani, M. Aljabri and R. A. Mustafa Mohammad “Ad Click Fraud Detection Using Machine Learning and Deep Learning Algorithms” in IEEE Access, vol. 13, pp. 12746-12763, 2025.

In the world of internet advertising, click fraud is still a major worry since it squanders advertising dollars and compromises the integrity of digital marketing ecosystems by diverting money from legitimate advertisers. A recent study investigated this problem by using a thorough feature engineering and extraction process designed to find minute behavioral patterns that distinguish authentic clicks from fake ones. The study assessed the efficacy of nine different Deep Learning (DL) and Machine Learning (ML) models in identifying click fraud. ML models showed remarkable accuracy after Recursive Feature Elimination (RFE) was applied. Decision Tree (DT) and Random Forest (RF) models revealed accuracy rates over 98.99%, while Gradient Boosting (GB), LightGBM, and XGBoost models recorded accuracy rates exceeding 98.90%. Models such as Artificial Neural Networks (ANN) also showed high precision scores, above 98%, demonstrating their capacity to correctly detect fraudulent behavior. Furthermore, DL models with high performance included Convolutional Neural Network (CNN), Deep Neural Network (DNN), and Recurrent Neural Network (RNN); the RNN achieved an accuracy of 97.34%. The study provides important insights for the creation of efficient anti-fraud measures in the online advertising sector and demonstrates the superiority of tree-based and advanced learning algorithms in identifying click fraud.

**[2] Clicking Fraud Detection for online advertising using Machine Learning
Vandavasi Baba Mahesh; Koneru Venkata Sai Chandra; Lammata Shiva Prasad Babu;
Velagal Aarthi Sowjanya; Dr Moulana Mohammed 2024**

Ad agencies have turned their attention to online and in-app advertising as a result of the rise of social media and digital technology. Online advertising is a major source of revenue for advertising networks that act as a go-between for marketers and companies. Ad networks use a pay-per-click (PPC) payment model, which is based on how many times advertisers click on their advertising. However, click fraud is an increasing security risk associated with this payment method. The unlawful activity of clicking on any paid advertisement in an attempt to boost publishers' profits or hurt advertisers' bottom lines is known as click fraud. In several fields of study, including cybersecurity, artificial intelligence is being utilized more and more to tackle difficult problems and produce surprising outcomes. In order to identify if a user is human or a bot, this article created a number of machine learning models and used a set of analytical concepts to compare performance.

**[3] Click Fraud Detection of Online Advertising Using Machine Learning Algorithms
Benjamin Kirkwood; Mounika Vanamala; Naeem Seliya 2024 IEEE International
Conference on Electro Information Technology (eIT)**

Click fraud has grown to be a significant worry for businesses as internet advertising continues to expand rapidly. Advertisers pay publishers for every click on their ad on the publisher's website. Deliberately clicking on internet advertisements with the intention of defrauding the website host or depleting the advertiser's budget is known as click fraud. Because online advertising is so lucrative, both of these factors make click fraud a tempting strategy for scammers. Scripts that click on these ads regularly are the source of these false clicks. There are ways to determine whether click fraud is taking place by looking at the intervals between clicks on a particular advertisement. In order to help marketers determine which clicks are fake, this study suggests a machine learning-based method for click fraud detection. Numerous Machine Learning algorithms, including logistic regression, random

forest, and neural networks, are used in the suggested method. The TalkingData Ad Tracking Fraud Detection click log dataset is used to train these models.

[4] GemmaWithLoRA: A New Approach to Click Fraud Detection Bingzhou Dai; Lei Wang; Xiaoyong Zhao; Yu Guo; Mohammad S. Obaidat 2024 International Conference on Communications, Computing, Cybersecurity, and Informatics (CCCI)

Pre-trained large language models (LLMs) have proven to have strong application potential in a variety of industries as artificial intelligence technology advances quickly. In the pay-per-click (PPC) paradigm, where fraudulent activities seriously reduce the efficiency of ads and provide security problems, click fraud is a major problem in the digital advertising ecosystem. Even though many conventional fraud detection techniques have attained a respectable level of accuracy, they continue to have significant feature engineering problems and poor generalization. This study investigates the use of LLMs for click fraud detection by utilizing LoRA fine-tuning technologies in conjunction with the Gemma-2b model's strong feature extraction capabilities and excellent generalization performance. The effectiveness of LLMs in click fraud detection tasks was demonstrated by extensive testing conducted on the TalkingData2017 dataset, which revealed that accuracy was attained at 83%.

[5] Detecting Fraud in Arabic Online Advertisements using Deep Learning Techniques for Image Classification Marouane Dirchaoui; Abdallah Abarda 2024 International Conference on Innovation and Intelligence for Informatics, Computing, and Technologies (3ICT)

For marketers looking to connect with people on social media, online ads are an essential tool. However, fraudsters also take use of them to initiate phishing assaults. The goal of this study is to use deep learning methods for picture classification to identify fraudulent Arabic online ads on social media sites. With 914 photos, 457 of which are real and 457 of which are fake, we created the first dataset of Arabic fraudulent advertisements. The performance of a convolutional neural network model and two transfer learning models, MobileNetV2 and NASNetMobile, is assessed in this study. According to our findings, the models perform noticeably better when the image size is increased from 150 pixels to 224 pixels. Specifically,

the NASNetMobile model obtained an F1 score of 87.43%, accuracy of 87.43%, precision of 90.91%, and recall of 84.21%. With a false positive rate of 9.09% and a false negative rate of 15.79%, the model showed a balanced detection capability, successfully differentiating between real and fraudulent advertisements in spite of their similarities. Future studies will concentrate on building sophisticated models to handle both formats and enlarging the dataset to include both images and videos of advertisements.

[6] Click Fraud Detection of Online Advertising–LSH Based Tensor Recovery Mechanism Fumin Zhu; Chen Zhang; Zunxin Zheng; Sattam Al Otaibi IEEE Transactions on Intelligent Transportation Systems, 2022

Online advertising uses the Internet as a means of reaching promotional consumers with marketing messaging. In order to meet the growing demands of internet commerce, it has been expanding recently. Every time a prospective customer clicks on the advertisement, advertisers bid and pay for it. Malicious clicks that imitate actual user behavior can fool the Pay-Per-Click (PPC) system into interpreting their clicks as authentic. Advertisers suffer enormous financial losses as a result, and internet advertising platforms' credibility is severely damaged. Common methods for identifying fraudulent clicks include data interpretation using machine learning models and dynamic customization. Exploring inherent relationships among a sequence of data is different with these methods since they treat multi-dimensional data as a single feature vector or matrix. The existing fraud prediction system has a relatively low efficiency due to the millions of frauds clicks on different types that occur every day, which further scatters the focus of models. We present a tensor-based approach to forecast fraud clicks in order to address the fraud click problem. In order to investigate hidden information beneath each data set and the combined effect among a sequence of data, this work examined reconstructing data into a high-rank tensor and implementing tensor decomposition and transformation. Comprehensive tests are conducted utilizing real-world data to evaluate the suggested tensor transformation technique with locality-sensitive hashing (LSH). Our model can perform significantly better in terms of accuracy and prediction-recall rate when compared to the most advanced machine learning algorithms.

[7] An Intelligent System for Identifying Fake Job Ads Using CNN-BiGRU and CNN-BiLSTM Sejal Badere; Tanvi Daf; Sakhi Chetule; Sakshi Barange; Aniruddha Auchat; Nikunj Jadhav; Sushil Chavhan 2024

Online job portals are now a common way for job seekers to locate employment opportunities in the current digital era. However, the number of fraudulent job advertisements has made it harder for job seekers to spot legitimate openings, and only a small number of models are able to accurately forecast the outcomes. Using deep learning approaches, such as CNN-BiGRU (Bidirectional Gated Recurrent Units) and CNN-BiLSTM (Bidirectional Long Short-Term Memory), in conjunction with Natural Language Processing (NLP) methods like tokenization and count vectorization, our study attempts to address this problem.

We gathered data from Google Dataset Search in order to train our model. After a great deal of testing and optimization, we found that CNN-BiGRU and CNN-BiLSTM were quite good at differentiating between real and fake job postings. In order to give job seekers a safer and more secure online platform experience, our model proved to be highly reliable in identifying fake job posts. It achieved an accuracy rate of 97% in removing fraudulent job advertisements, a significant improvement of +5% in accuracy. This has far-reaching implications for the development of fraud detection systems in various domains, including the Digital Employment Market, Government Agencies, Social Media Platforms, Research, and Policy Development.

[8] Reveal: Online Fake Job Advert Detection Application using Machine Learning C Prashanth, Deepanjali Chandrasekaran, Bhuvanashree Pandian, Kavitha Duraipandian, Thomas Chen, Mithileysh Sathiyarayanan, 2022

Rapid technological advancements are being made to combat the rise in employment scams. Because online job scams are difficult to identify, the offenders have plenty of time to leave the location where the crime was done. As a result, they may be in a different nation, far from the site of the crime, by the time it is discovered. Such incidents, in which a specific individual is targeted, are common in today's digital environment. Unprecedented levels of knowledge dissemination in human history were made possible by the advent of the internet

and the ease of access to social networking sites like Instagram and Twitter. Because people are weak and easily tricked, technology advancements are insufficient to prevent online job scams. False job openings are promoted to lure applicants so that scammers can obtain personal data such home address, email address, phone number, birthdate, past employment history, bank account information, and full identity. In order to help candidates be cautious when applying for jobs that are legitimate and trustworthy, we created Reveal, a machine learning-based web tool, in this study.

CHAPTER 3

PROBLEM OVERVIEW

3.1 PROBLEM IDENTIFICATION

Online advertising has emerged as a key element of marketing strategy in the constantly changing digital landscape, allowing companies to efficiently and rapidly reach large audiences. However, there are now significant security flaws as a result of the increase in internet advertisements. Phishing and malicious ads have become a serious issue, posing as trustworthy advertisements while delivering damaging content to unwary consumers. In addition to endangering user safety, these dishonest advertisements harm the reputation of trustworthy companies and advertising networks. There are still a number of significant issues facing digital advertising today:

Proliferation of Malicious and Phishing Advertisements:

Phishing links, spyware, or deceptive content are commonly inserted by cybercriminals into advertisements that look like real advertising content.

- These advertisements cause dangerous downloads or send visitors to phony login portals or scam websites.
- These advertisements frequently get past simple ad filters and are challenging to manually identify; victims may unintentionally divulge private information, which could result in identity theft or financial fraud.

Lack of Real-Time Detection Mechanisms:

To find dangerous ads, the majority of current systems use delayed reporting or static blacklists.

- Initial exposure cannot be avoided by these reactive techniques.
- End users do not have access to proactive, real-time ad scanning technologies.

Limited Integration of Machine Learning for Threat Detection:

Although machine learning has shown promise in detecting cyberthreats, its use in ad fraud is still relatively new.

- Few platforms identify ad content dynamically using machine learning methods.
- Existing solutions don't adapt to changing attacker tactics or continuously learn from new threats.
- For browser-based deployment, there are no sophisticated models that can provide real-time user protection.

Impact on User Trust and Platform Credibility:

The proliferation of misleading advertisements erodes confidence in ad networks and digital platforms.

- Users might steer clear of websites where they have seen dangerous advertisements in the past.
- Because of user distrust, legitimate marketers experience a decline in interaction.
- Online advertising's reputation is damaged overall, which has an impact on compliance and income sources.

Lack of User-Friendly Security Tools:

Most people do not have easy-to-use methods to protect themselves from harmful advertisements.

Currently available ad blockers and browser plugins are general and lack intelligent classification.

- There aren't many simple, lightweight options for protecting your daily browsing.
- Typically, machine learning tools are too complex or resource-intensive for regular users.

3.2 PROBLEM STATEMENT

Online ads are an effective way for businesses to reach consumers all over the world in the current digital ecosystem. However, fraudsters are increasingly taking advantage of this area by hiding dangerous content—like malware, phishing links, or fake images—into what appear to be genuine ads. These fraudulent advertisements trick people into clicking on dangerous links, which can lead to privacy violations, data breaches, and financial loss.

Such fake advertisements not only hurt specific users but also compromise the credibility of digital advertising networks and cost real advertisers a lot of money. Static blacklists and simple filters, two reactive detection tools, are unable to keep up with the constantly changing tactics utilized in ad fraud.

A sophisticated, real-time system that can recognize and stop dangerous or fraudulent ads before they reach users is required to address this pressing problem. This research suggests a machine learning-based method for identifying fraudulent activity by analyzing ad data.

3.3 OBJECTIVE

In order to improve consumer security and confidence in digital advertising platforms, the main objective of this project is to create a machine learning and deep learning-based system that can recognize and notify users in real time of potentially fraudulent adverts.

Phishing and Fraudulent Ad Detection:

Use sophisticated machine learning algorithms to analyze content patterns and URLs in order to detect phishing ad links, fake image advertising, and deceptive video ads.

Fraudulent Click Classification:

Classify ad clicks as genuine or fraudulent using machine learning and deep learning approaches to accurately identify malicious ads.

Real-Time Fraud Detection via Flask API:

Installing a Flask API will allow for real-time fraud detection, speedy and effective ad data processing, and timely warning generation.

Browser Extension for User Alerts:

Connect the system to a browser extension that checks ads and alerts users to potentially hazardous or fraudulent ones, enabling them to stay away from dangerous information.

3.4 METHODOLOGY

This project consists of 5 modules

- Dataset Collection
- Dataset Cleaning and Feature Extraction
- Model Training and Evaluation
- Developing Browser Extension
- Integrating Browser Extension and Model

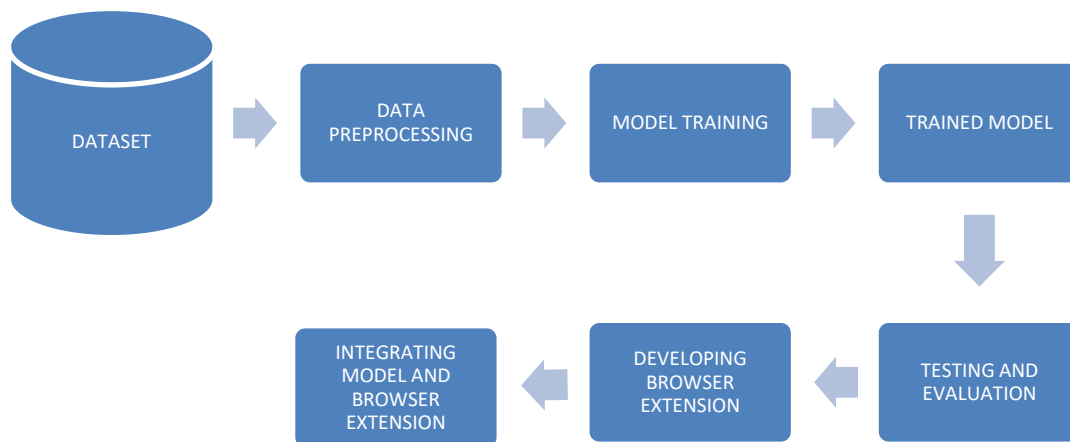


Figure 3.4.1 Block Diagram of Phishing Ads Detector

1. Data Collection

Gathering a thorough and organized dataset of links to both genuine and phishing advertisements was the first stage of the research. In order to guarantee efficient model training and precise classification, the dataset has to have a balanced representation of dangerous and secure URLs in addition to pertinent attributes that mirror typical phishing

attempt patterns.

1.1 Data Source:

There are 235,795 URLs in the sample, of which 134,850 are authentic and 100,945 are phishing. Since the majority of the URLs in the dataset are current, it is extremely valuable for identifying contemporary phishing tactics.

1.2 Contents:

100,945 phishing URLs and 134,850 authentic URLs make up the dataset's total of 235,795 URLs. Since the majority of the included URLs are recent, the dataset is extremely pertinent for identifying contemporary phishing tactics.

1.3 Feature Representation:

The associated web pages' source code and URL structure were both examined for features. These characteristics aid in identifying structural and behavioral trends frequently observed in phishing advertisements. Among the noteworthy aspects are:

- The URLCharProb analyzes the probability distribution of characters in the URL.
- The URLTitleMatchScore compares the page title with the domain to identify inconsistencies.
- The CharContinuationRate measures character repetition or unusual sequences in the URL.
- The TLDLegitimateProb estimates the legitimacy based on the top-level domain (TLD).

2. Data Preprocessing

Cleaning the dataset and extracting valuable features for model training came next once it was collected.

2.1 Data Cleaning:

- To avoid biased learning, duplicate records were eliminated. If necessary, rows were discarded, or mean or median imputation was used to address missing values.

- To maintain uniformity, URL fields were normalized by changing the content to lowercase and removing any extraneous spaces or characters.

2.2 Feature Extraction:

- To assist the algorithm in identifying phishing tendencies, pertinent features were taken from every URL.
- These consist of the URL length, the number of dots (.), and the slashes (/); larger or more complicated URLs are frequently a sign of questionable activity.
- HTTPS is present; trustworthy websites usually employ secure protocols.
- Top-Level Domain (TLD) — encoded numerically using LabelEncoder to help detect uncommon or less often used domains.
- Commonly utilized in malicious or obfuscated URLs include special characters, the amount of numbers, and equals signs (=).

3. Model Training

A. Extreme Gradient Boost (XG Boost):

The XGBoost classifier was chosen for its effectiveness in detecting harmful advertisements and phishing because of its ability to handle imbalanced datasets and its efficiency with structured data. Through the use of regularization strategies like L1 and L2 to lessen overfitting and the capture of non-linear patterns, it provides great accuracy. variables taken from ad URLs, such as URL length, number of dots or slashes, HTTPS presence, top-level domain (TLD), special characters, digit count, and use of symbols like "=", were used to train the model. These variables are all suggestive of phishing behavior. For improved precision and recall, the model was modified using gradient descent-based optimization and cross-validation after starting with default hyperparameters. To improve computing efficiency and expedite evaluation, the dataset was split up during training, even though XGBoost doesn't rely on conventional batch processing.

B. K-Nearest Neighbors (KNN) Algorithm

KNN is an easy-to-understand algorithm that is effective as a baseline model for phishing detection. It makes predictions based on the majority class of the closest neighbors and classifies data by calculating the distance (such as Euclidean) between data points. Predictions may be skewed if features are not appropriately normalized, as feature scaling is a major factor in KNN performance. Furthermore, because each prediction in KNN needs a distance calculation, it might be computationally wasteful, particularly in real-time applications. By contrast, XGBoost creates a model in advance, enabling predictions that are significantly quicker and more accurate, which makes it a superior option for phishing detection.

3.5 PLAN OF EXPERIMENTS

The purpose of the phishing ad detection system's experimental strategy is to verify the basic components' performance, functionality, and real-time responsiveness. To guarantee smooth integration, efficient ad classification, and timely user notification, the system elements are tested both separately and collectively. Determining input, examining processing logic, and contrasting output with anticipated outcomes are all part of each test case.

1. Dataset Preparation and Preprocessing

Gathering a large dataset of ad URLs, including both authentic and phishing/malicious links, was the initial step. Relevant variables like URL length, HTTPS presence, amount of special characters, and domain structure were extracted from the dataset through analysis. To get the data ready for model training, label encoding and normalization methods were used. This test made sure that the features that were extracted accurately reflected the traits of deceptive ads.

2. Model Training and Validation

The preprocessed dataset was used to train the XGBoost classifier. Hyperparameters like learning rate, maximum depth, and regularization strength were adjusted throughout training trials. To make sure the model performs properly when applied to new data,

cross-validation was employed. F1-score, recall, accuracy, and precision were used to assess the model's performance. The experiment demonstrated that XGBoost was able to classify fake ad links with high accuracy and precision.

3. Flask API Deployment

Flask was used to deliver the trained model as a RESTful API. The test examined the API's capacity to receive input URLs, process them instantly, and provide classification results with the least amount of latency. The accuracy, speed, and fault tolerance of the endpoint answers under concurrent queries were verified. This stage used a scalable backend service to guarantee real-time detection capability.

4. Browser Extension Integration

The browser extension's capacity to scan online advertisements, record ad URLs, and interact with the Flask API for classification was evaluated. When the extension recognized a potentially harmful or phishing ad, it presented warnings. This test confirmed timely user notifications through pop-up messages or warning icons, as well as seamless frontend-backend interaction.

5. Real-Time Threat Detection Simulation

Web pages with links to both safe and phishing advertisements were seen while the extension was enabled in order to mimic real-world situations. The system's capacity to identify hidden links in picture and video advertisements was put to the test. In order to maintain usability, the experiment verified precise classification in real-time and made sure the extension merely alerted the user rather than blocking advertisements.

6. User Experience Testing

The extension's UI/UX was evaluated for overall system responsiveness, warning message clarity, and usability. The message formatting, alert color-coding, and warning placement were improved based on input from simulated user sessions. During this stage, it was made sure the instrument was user-friendly and beneficial without being invasive.

7. Security and Privacy Validation

The system's ability to handle data securely was evaluated, guaranteeing that user surfing activity was neither saved nor exploited. To guard against tampering or eavesdropping, API communications were tested over HTTPS. This test confirmed that the plugin only served as a safety measure.

3.6 SYSTEM SPECIFICATIONS

HARDWARE REQUIREMENTS

- Operating System: Windows 7 or above
- RAM: 6 GB or above
- Processor: Intel i3, Ryzen

SOFTWARE REQUIREMENTS

- Web Browser - Chrome (latest), Firefox (latest), Edge
- Backend Runtime - Node.js (Server Actions)
- Package Manager - npm
- Developmnt Tools - VS Code

FRONTEND TECHNOLOGY

JAVASCRIPT

The main technology utilized to construct the Ad Fraud Detection system's frontend is JavaScript. Through smooth browser extension capability and real-time content updates, it facilitates dynamic user-system interaction. In order to detect fraudulent behavior, JavaScript manages the scanning of ads, records user events such as ad clicks, and interfaces with the Flask API backend. Fraud alerts or warnings can be displayed instantly without a page refresh because of its ability to change the DOM. JavaScript also improves responsiveness and usability by highlighting questionable advertisements and displaying detection results straight in the user's browser. JavaScript is a great option for interacting with browser environments

and guaranteeing seamless real-time fraud detection because of its lightweight and adaptable nature.

BACKEND TECHNOLOGY

FLASK API

A lightweight Python web framework called Flask forms the backend of the Ad Fraud Detection system. Real-time communication between the machine learning models trained to identify fraudulent adverts and the frontend (JavaScript/browser extension) is its responsibility. When a user interacts with an advertisement, the Flask API receives the data from the frontend (such as the ad link or metadata), evaluates the request, applies the classification model, and provides a response indicating whether the ad is authentic or fake. Flask's ease of use and adaptability make it perfect for quick prototyping and ML model integration by facilitating the development of RESTful endpoints.

CHAPTER 4

DESIGN AND MODELLING

4.1 Ad Analysis Interface and URL Monitoring

The browser extension's smooth integration into the user's browsing environment is the first step in the user experience. With a focus on examining ad links, banner pictures, and embedded video content, this plugin continuously tracks ads that show up on websites. Without interfering with user engagement, it instantly retrieves the ad's target URL, title, and metadata. The first layer of data gathering is formed by these inputs, after which it is sent to the backend system for additional analysis.

4.2 Feature Extraction and Preprocessing

The system extracts feature from the ad links after capturing the advertisement data. Analyzing URL attributes such length, depth, HTTPS usage, special character presence, domain age, and top-level domain information is part of this. Ad text is also analyzed for trends typical of deceptive or fraudulent ads. After being extracted, these features are preprocessed (normalized, encoded, and filtered) to produce a structured input that meets the requirements of the trained model.

4.3 Phishing Ad Detection using Machine Learning

The trained XGBoost model, which has been optimized to classify ad links as safe or fraudulent, handles the core of the fraud detection logic. A carefully selected dataset of authentic and fraudulent URLs was used to train the model. The model receives the preprocessed feature set upon detection and provides a classification and a probability score that indicates the prediction's level of confidence. As soon as an ad is identified, its prediction pipeline ensures an instant response because it operates in almost real-time.

4.4 Flask API Integration

The browser extension and the machine learning model communicate with each other over a lightweight Flask-based REST API. Requests from the extension in JSON format are received by the API, which then sends the ad data to the model and returns categorization results. To ensure a smooth surfing experience, the system is tuned for low-latency replies. Additionally, it records questionable activity for future study and model retraining.

4.5 Real-Time User Notification System

The browser extension alerts the user with a warning banner or popup when it detects a fraudulent or questionable advertisement. In addition to explicitly outlining the risk, this notification can provide choices to report, ignore, or find out more about the possible fraud. Clarity and non-intrusiveness are given top priority in the design, guaranteeing that people stay informed without feeling overloaded.

4.6 Visual Analytics and Detection Logs

The system has a logging and visualization component that monitors the quantity of scanned advertising, reported frauds, and detection accuracy over time in order to provide insights into ad fraud trends. This module can be expanded in the future to give end users dashboards that display fraud statistics, common fraud sources, and current protection status, even if its primary purpose is development assessment.

4.7 Scalability and Browser Compatibility

The architecture of the entire system prioritizes compatibility and scalability. Modern web browsers like Chrome and Firefox are supported by the browser extension, which is also sufficiently modular to enable future integration with mobile browsers. By containerizing the Flask API and running it on cloud platforms such as AWS or Render, the backend can be expanded to users.

CHAPTER 5

RESULTS AND DISCUSSIONS

5.1 Evaluation Metrics used

Evaluation measures like accuracy, precision, recall, and F1-score were employed to assess the system's efficacy in identifying fake adverts. These metrics offered a thorough evaluation of the model's accuracy in identifying both authentic and fraudulent advertisements. Measurements of real-time detection latency were also made to make sure the system could react fast without interfering with user experience.

5.2 Performance of the XGBoost Classifier

The XGBoost model demonstrated a strong performance, achieving over 99% accuracy in detecting phishing and malicious ad links. It handled imbalanced datasets effectively and captured complex patterns using engineered features such as URL length, HTTPS presence, and special character frequency. With precision, recall, and F1-scores all exceeding 90%, the model proved reliable in minimizing both false positives and false negatives.

5.3 Effectiveness of Real-Time Detection

Real-time detection of questionable advertisements was made possible via the Flask API, which also effortlessly integrated with the browser extension. From content scanning to warning generation, the entire pipeline kept response times under two seconds. This made it possible to send out user alerts instantly without interfering with regular online browsing. Across image advertisements, video-based promos, and text-based links, the system reliably provided accurate fraud warnings.

5.4 Browser Extension Integration and User Notification

The system's user-facing element was the browser extension. It successfully communicated with the backend API for classification and extracted ad-related data from webpages. It assisted consumers in making educated judgments before dealing with potentially hazardous advertisements by displaying non-intrusive warning messages when

it detected fraudulent content. The extension worked flawlessly on a variety of websites, demonstrating its dependability and cross-platform compatibility.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

1.1 CONCLUSION

The project's ad fraud detection technology successfully tackles the escalating issues around false digital ads. It uses a combination of machine learning and deep learning algorithms to detect misleading video content, phishing ad links, and deceptive picture adverts. The XGBoost model was selected because it performs well on both balanced and structured datasets, which makes it ideal for malicious ad classification. Carefully designed elements like domain indicators and URL structures—known warning signs for phishing attempts—were used to educate it.

The model was implemented using a Flask-based API that works in unison with a browser extension to improve user safety in real-time. The solution puts the user experience first by promptly and clearly alerting users to potentially dangerous adverts rather than completely censoring them. By enabling people to make knowledgeable decisions while browsing, this non-intrusive method promotes awareness and confidence.

Overall, the system's capacity to detect ad fraud with high accuracy and low latency validates its dependability and relevance. It is a reliable, scalable, and flexible way to safeguard users and preserve the integrity of online advertising spaces. Using AI-driven technologies, the initiative shows a successful step toward automated identification and prevention of digital ad fraud.

1.2 FUTURE SCOPE

1. Integration of Deep Learning for Multimedia Analysis

Textual and URL-based elements are the main emphasis of the present system. Convolutional Neural Networks (CNNs), one of the most sophisticated deep learning models, can be used in the future to more precisely analyze the content of picture and video advertisements. Deeper examination and the identification of visually misleading features or improper content that might evade conventional filtering systems would be made possible by this.

2. User Feedback and Reinforcement Learning

Including a feedback mechanism that allows users to flag advertisements as "safe" or "suspicious" will aid in the model's ongoing training and accuracy improvement. Reinforcement learning techniques can make use of this real-time feedback loop to dynamically adjust the system to newly discovered fraud patterns and user behavior.

3. Multilingual and Cross-Regional Detection

Expanding the detection capabilities to support different languages will improve the system's usefulness for viewers around the world, since ad fraud is not restricted to a single location or language. To identify fraud in localized or regionalized ad content, language-based natural language processing algorithms can be used.

4. Pre-Launch Ad Screening for Advertisers and Platforms

The technology can be expanded to provide advertisers or ad platforms with fraud detection tools directly. By enabling pre-screening of ad campaigns prior to publication, this would lower the likelihood that end users would see fraudulent advertisements. Ad tech businesses could be able to purchase it as (SaaS).

5. Dashboard and Analytics Module

To display metrics like the quantity of risks found, fraud types, browsing safety score, and often flagged URLs, a dashboard that is easy to use might be created. End users and platform

managers would both gain from this module's insights regarding the type and frequency of fraud attempts that have been recognized.

CHAPTER 7

REFERENCES

- [1] R. A. Alzahrani, M. Aljabri and R. A. Mustafa Mohammad, “Ad Click Fraud Detection Using Machine Learning and Deep Learning Algorithms” in IEEE Access, vol. 13, pp. 12746-12763, 2025.
- [2] Neeraja, Anupam, Sriram, Subhani Shaik and V. Kakulapati, “Fraud Detection of Ad Clicks Using Machine Learning Techniques” on Journal of Scientific Research and Reports, Volume 29, Issue 7, Page 84-89, 2023.
- [3] B. Kirkwood, M. Vanamala and N. Seliya, “Click Fraud Detection of Online Advertising Using Machine Learning Algorithms” on IEEE International Conference on Electro Information Technology, Eau Claire, WI, USA, 2024.
- [4] Clicking Fraud Detection for online advertising using Machine Learning Vandavasi Baba Mahesh; Koneru Venkata Sai Chandra; Lammata Shiva Prasad Babu; Velagal Aarthi Sowjanya; Dr Moulana Mohammed 2024
- [5] GemmaWithLoRA: A New Approach to Click Fraud Detection Bingzhou Dai; Lei Wang; Xiaoyong Zhao; Yu Guo; Mohammad S. Obaidat 2024 International Conference on Communications, Computing, Cybersecurity, and Informatics (CCCI).
- [6] Detecting Fraud in Arabic Online Advertisements using Deep Learning Techniques for Image Classification Marouane Dirchaoui; Abdallah Abarda 2024 International Conference on Innovation and Intelligence for Informatics, Computing, and Technologies (3ICT)
- [7] Malak Aljabri, Rami Mustafa, A. Mohammad, “Click fraud detection for online advertising using machine learning” on Egyptian Informatics Journal Volume 24, Issue 2, July 2023.

- [8] Click Fraud Detection of Online Advertising–LSH Based Tensor Recovery Mechanism
Fumin Zhu; Chen Zhang; Zunxin Zheng; Sattam Al Otaibi IEEE Transactions on Intelligent Transportation Systems, 2022
- [9] An Intelligent System for Identifying Fake Job Ads Using CNN-BiGRU and CNN-BiLSTM
Sejal Badere; Tanvi Daf; Sakhi Chetule; Sakshi Barange; Aniruddha Auchat; Nikunj Jadhav; Sushil Chavhan 2024
- [10] S. Shaik and V. Kakulapati, "Fraud detection of AD clicks using machine learning techniques", J. Sci. Res. Rep., vol. 29, no. 7, pp. 84-89, Jun. 2023.
- [11] L. Singh, D. Sisodia, K. Shashvat, A. Kaur and P. C. Sharma, "A reliable click-fraud detection system for the investigation of fraudulent publishers in online advertising" in Applied Intelligence in Human-Computer Interaction, Boca Raton, FL, USA: CRC Press, Jul. 2023.

ANNEXURE I

SOURCE CODE

App.py

```
from flask import Flask, request, jsonify
import numpy as np
import pandas as pd
import tensorflow as tf
import joblib
from tensorflow.keras.models import load_model
from sklearn.preprocessing import LabelEncoder
from urllib.parse import urlparse
from flask_cors import CORS
import re

# Load the trained model
MODEL_PATH = "models/phishing_nn_model.h5"
model = load_model(MODEL_PATH)

# Load the label encoder for TLD conversion
label_encoder = joblib.load("models/tld_label_encoder.pkl")

app = Flask(__name__)
CORS(app)
def extract_features(url):
    parsed_url = urlparse(url)
    tld = parsed_url.netloc.split('.')[1] if '.' in parsed_url.netloc else ""
    return {
        "url_length": len(url),
        "num_dots": url.count('.'),
        "num_hyphens": url.count('-'),
        "num_slashes": url.count('/'),
        "num_digits": sum(c.isdigit() for c in url),
        "has_https": 1 if parsed_url.scheme == "https" else 0,
        "domain_length": len(parsed_url.netloc),
        "subdomain_count": parsed_url.netloc.count('.'),
        "special_chars": len(re.findall(r"[@!#$%^&*()+=]", url)),
        "TLD_length": len(tld),
        "num_equals": url.count('='),
        "TLD_numeric": label_encoder.transform([parsed_url.netloc.split('.')[1]])[0] if
        parsed_url.netloc.split('.')[1] in label_encoder.classes_ else -1
    }
```

```

@app.route("/predict", methods=["POST"])
def predict():
    try:
        data = request.json
        url = data.get("url")

        if not url:
            return jsonify({"error": "No URL provided"}), 400

        features = extract_features(url)
        input_data = np.array(list(features.values())).reshape(1, -1)
        prediction = model.predict(input_data)[0][0]
        result = "Phishing" if prediction > 0.5 else "Legitimate"

        return jsonify({"url": url, "prediction": result, "confidence": float(prediction)})

    except Exception as e:
        return jsonify({"error": str(e)}), 500

if __name__ == "__main__":
    app.run(debug=True)

```

Train_XGBoost.py

```
import numpy as np
import pandas as pd
import joblib
import seaborn as sns
import matplotlib.pyplot as plt
import xgboost as xgb
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score, recall_score, precision_score, f1_score, ConfusionMatrixDisplay
)
from sklearn.neighbors import KNeighborsClassifier

# Load dataset
DATASET_PATH = "/content/PhiUSIIL_Phishing_URL_Dataset.csv"
df = pd.read_csv(DATASET_PATH)

# Drop unnecessary features
columns_to_drop = [
    'URL', 'Domain', 'Title', 'URLLength', 'DomainLength', 'IsDomainIP', 'CharContinuationRate',
    'TLDDegitimateProb', 'URLCharProb', 'TLDDLength', 'NoOfSubDomain', 'HasObfuscation',
    'NoOfObfuscatedChar', 'ObfuscationRatio', 'NoOfLettersInURL', 'LetterRatioInURL',
    'NoOfDegitsInURL',
    'DegitRatioInURL', 'NoOfEqualsInURL', 'NoOfQMarkInURL', 'NoOfAmpersandInURL',
    'NoOfOtherSpecialCharsInURL',
    'LineOfCode', 'LargestLineLength', 'HasTitle', 'HasFavicon', 'Robots', 'NoOfURLRedirect',
    'NoOfSelfRedirect',
    'NoOfPopup', 'NoOfiFrame', 'HasExternalFormSubmit', 'HasPasswordField', 'Bank', 'Pay',
    'Crypto', 'NoOfImage',
    'NoOfCSS', 'NoOfJS', 'NoOfSelfRef', 'NoOfEmptyRef', 'NoOfExternalRef'
]
df.drop(columns_to_drop, axis=1, inplace=True)

# Separate features and target
X = df.drop('label', axis=1)
y = df['label']
```

```

# Encode TLD column
label_encoder = LabelEncoder()
X['TLD_numeric'] = label_encoder.fit_transform(X['TLD'])
X.drop(['TLD'], axis=1, inplace=True)

# Save label encoder
joblib.dump(label_encoder, "/content/tld_label_encoder.pkl")

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# =====
# XGBoost Model
# =====
dtrain = xgb.DMatrix(X_train, label=y_train)
dtest = xgb.DMatrix(X_test, label=y_test)

params = {
    'objective': 'binary:logistic',
    'eval_metric': ['logloss', 'error'],
    'learning_rate': 0.1,
    'max_depth': 6,
    'seed': 42
}
evals = [(dtrain, 'train'), (dtest, 'test')]
evals_result = {}
xgb_model = xgb.train(
    params=params,
    dtrain=dtrain,
    num_boost_round=100,
    evals=evals,
    early_stopping_rounds=10,
    evals_result=evals_result,
    verbose_eval=True
)

```

```

# Save XGBoost model
xgb_model.save_model("phishing_xgboost_model.json")

# XGBoost predictions
y_pred_xgb_probs = xgb_model.predict(dtest)
y_pred_xgb = (y_pred_xgb_probs >= 0.5).astype(int)

# =====
# KNN Model
# =====
knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train, y_train)

# KNN predictions
y_pred_knn = knn_model.predict(X_test)
y_pred_knn_probs = knn_model.predict_proba(X_test)[:, 1]

# =====
# Evaluation Metrics
# =====
metrics_xgb = {
    "Model": "XGBoost",
    "Accuracy": accuracy_score(y_test, y_pred_xgb),
    "Recall": recall_score(y_test, y_pred_xgb),
    "Precision": precision_score(y_test, y_pred_xgb),
    "F1 Score": f1_score(y_test, y_pred_xgb)
}
metrics_knn = {
    "Model": "KNN",
    "Accuracy": accuracy_score(y_test, y_pred_knn),
    "Recall": recall_score(y_test, y_pred_knn),
    "Precision": precision_score(y_test, y_pred_knn),
    "F1 Score": f1_score(y_test, y_pred_knn)
}

```

Combine Metrics into DataFrame

```
comparison_df = pd.DataFrame([metrics_xgb, metrics_knn])  
comparison_df.set_index('Model', inplace=True)
```

Display Comparison Table

```
print("\nModel Performance Comparison:\n")  
print(comparison_df)
```

Plot Comparison Bar Chart

```
plt.figure(figsize=(10, 6))  
comparison_df.plot(kind='bar', figsize=(10, 6), colormap='Set2')  
plt.title('Model Comparison: XGBoost vs KNN')  
plt.ylabel('Score')  
plt.xticks(rotation=0)  
plt.ylim(0.98, 1)  
plt.grid(axis='y')
```

```
plt.legend(loc='lower right')  
plt.tight_layout()  
plt.show()
```

Confusion Matrices for Both Models

```
sns.reset_orig()
```

```
print("\nXGBoost Confusion Matrix:")
```

```
ConfusionMatrixDisplay.from_predictions(y_test, y_pred_xgb, display_labels=['Legitimate',  
'Phishing'], cmap='Blues')
```

```
print("\nKNN Confusion Matrix:")
```

```
ConfusionMatrixDisplay.from_predictions(y_test, y_pred_knn, display_labels=['Legitimate',  
'Phishing'], cmap='Oranges')
```


Popup.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Phishing Ads Detector</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <h2>Phishing Ads Detector</h2>

  <!-- Toggle Button -->
  <div class="toggle-container">
    <button id="toggleButton" class="toggle off">OFF</button>
  </div>

  <p id="status">Status: OFF</p>

  <script src="popup.js"></script>
</body>
</html>
```

Style.css

```
body {
  font-family: Arial, sans-serif;
  text-align: center;
  padding: 20px;
  background: #002458; /* Dark Navy Background */
  color: white; /* Text color for contrast */
}

h2 {
  margin-bottom: 20px;
}

.toggle-container {
  display: flex;
  justify-content: center;
  align-items: center;
}

#toggleButton {
  width: 80px;
  height: 80px;
  border-radius: 50%;
  border: none;
  background-color: #00BFFF; /* Sky Blue for OFF */
  color: white;
  font-size: 16px;
  cursor: pointer;
  transition: all 2s ease; /* Smooth animation */
  box-shadow: 0 4px 10px rgba(0, 0, 0, 0.3);
}
```

```
#toggleButton.on {  
    background-color: #00BFFF; /* Sky Blue for ON */  
    transform: rotate(360deg) scale(1.1);  
}  
  
#toggleButton.off {  
    background-color: #00BFFF; /* Sky Blue for OFF */  
    transform: rotate(-360deg) scale(1.0);  
}  
  
#status {  
    margin-top: 20px;  
    font-weight: bold;  
    color: white;  
}
```

Popup.js

```
const toggleButton = document.getElementById("toggleButton");
const statusText = document.getElementById("status");

// Load state on popup open
chrome.storage.local.get(["detectionOn"], (result) => {
  const isOn = result.detectionOn || false;
  updateToggleButton(isOn);
});

// Toggle button click handler
toggleButton.addEventListener("click", () => {
  chrome.storage.local.get(["detectionOn"], (result) => {
    const isOn = result.detectionOn || false;
    const newState = !isOn;

    // Update storage
    chrome.storage.local.set({ detectionOn: newState }, () => {
      updateToggleButton(newState);

      // Inject or remove content script
      chrome.tabs.query({ active: true, currentWindow: true }, (tabs) => {
        if (newState) {
          chrome.scripting.executeScript({
            target: { tabId: tabs[0].id },
            files: ["content.js"]
          });
        }
      });
    });
  });
});
```

```

} else {
    // On OFF: refresh page to remove the effects (or implement cleanup inside
    content.js)
    chrome.tabs.reload(tabs[0].id);
    }
    });
    });
    });
});

function updateToggleButton(isOn) {
    if (isOn) {
        toggleButton.classList.remove("off");
        toggleButton.classList.add("on");
        toggleButton.textContent = "ON";
        statusText.textContent = "Status: ON";
    } else {
        toggleButton.classList.remove("on");
        toggleButton.classList.add("off");
        toggleButton.textContent = "OFF";
        statusText.textContent = "Status: OFF";
    }
}

```

Background.js

```
chrome.runtime.onInstalled.addListener(() => {  
  console.log("Phishing Ads Detector Installed");  
});
```

Content.js

```
(function() {  
  chrome.storage.local.get(["detectionOn"], (result) => {  
    if (!result.detectionOn) {  
      console.log("Phishing Ads Detector is OFF");  
      return;  
    }  
  
    console.log("Phishing Ads Detector is ON");  
  
    let ads = document.querySelectorAll("a[href*='ad'], iframe[src], img[src*='ad']");  
    let results = [];  
  
    ads.forEach(ad => {  
      let url = ad.href || ad.src;  
  
      fetch("http://localhost:5000/predict", {  
        method: "POST",  
        headers: { "Content-Type": "application/json" },  
        body: JSON.stringify({ url: url })  
      })  
        .then(response => response.json())  
        .then(data => {  
          if (data.prediction === "Phishing") {  
            ad.style.border = "3px solid red";  
            results.push(url);  
          }  
        })  
        .catch(error => console.error("Fetch error:", error));  
    });  
  });  
})
```

```

setTimeout(() => {
  if (results.length > 0) {
    alert(' ⚠ Warning: ${results.length} phishing ads detected on this page!');
    console.log("Phishing ads detected:", results);
  } else {
    alert(" ✅ No phishing ads detected.");
  }
}, 1000);
});
})();

```

Manifest.json

```

{
  "manifest_version": 3,
  "name": "Phishing Ads Detector",
  "version": "1.0",
  "description": "Detect phishing ads on websites.",
  "permissions": ["activeTab", "scripting", "storage"],
  "host_permissions": ["<all_urls>"],
  "background": {
    "service_worker": "background.js"
  },
  "action": {
    "default_popup": "popup.html",
    "default_icon": "icon.png"
  },
  "content_scripts": [
    {
      "matches": ["<all_urls>"],
      "js": ["content.js"]
    }
  ]
}

```

SNAPSHOTS

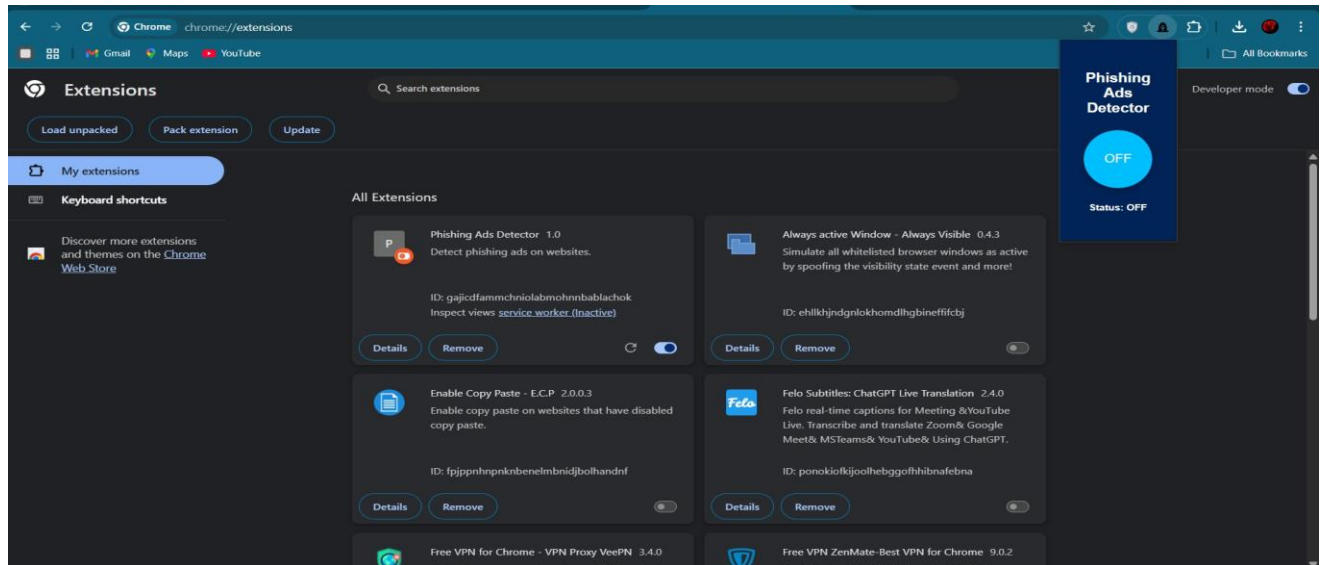


Figure 1.1 Extension UI of the Proposed System

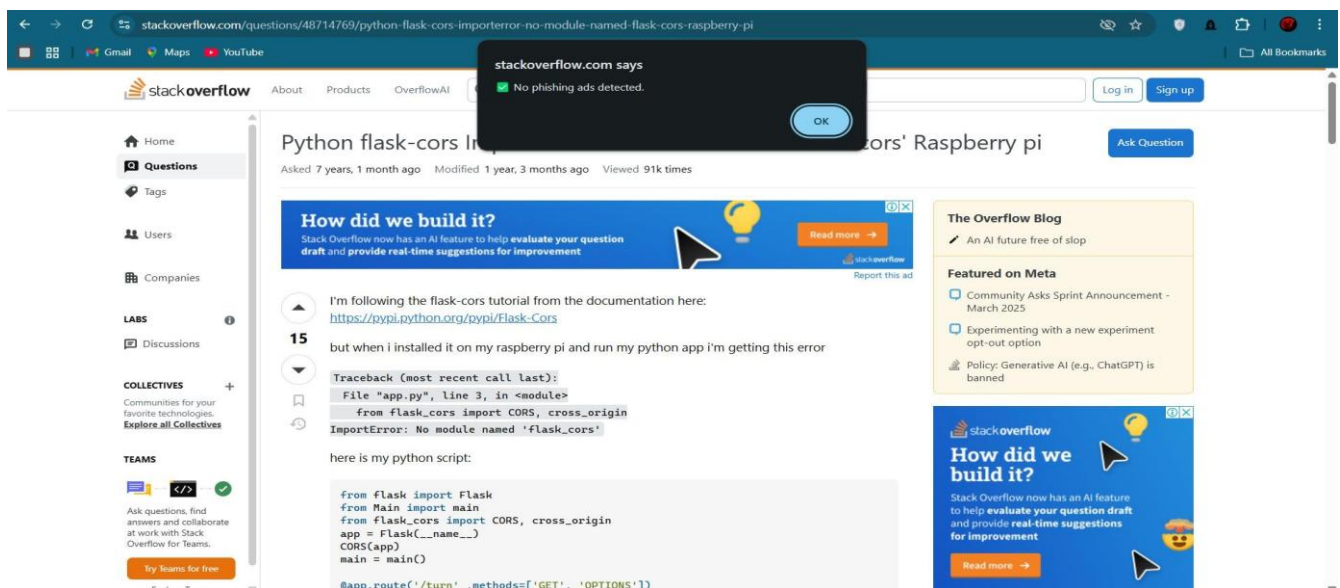


Figure 1.2 Popup UI of the Proposed System

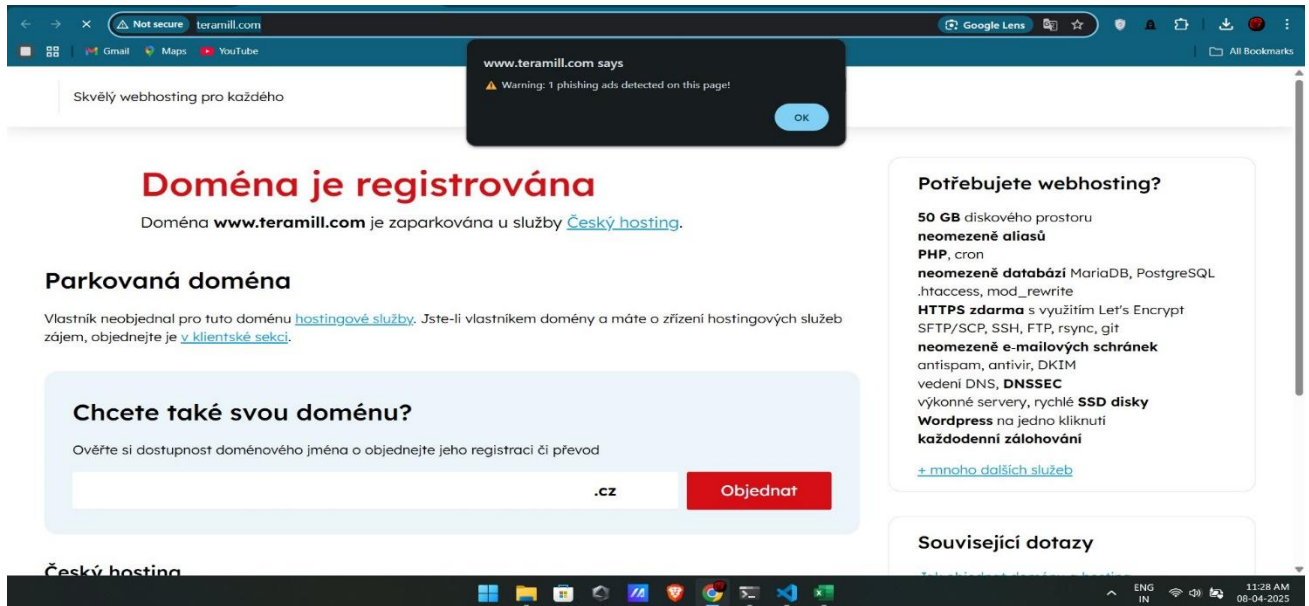


Figure 1.3 Output of the Proposed System